PHASE 2 · BEGINNER

DAY 6

Data Structures

Lists · Tuples · Dictionaries · Sets · Comprehensions · Slicing

■ Session Length	■ Difficulty	■ Deliverables
2 hours	Beginner → Core	3 files

30-Day Python Programming Course
Instructor Session Guide — Confidential Draft

1 · Session Overview

Goal

Students move beyond single values and learn to organise real collections of data using Python's four core built-in data structures. By the end of this session they will confidently choose between a list, tuple, dictionary, or set depending on the problem at hand, and can write comprehensions to build collections concisely.

Learning Outcome

By the end of Day 6, students can create, modify, and iterate over lists, tuples, dictionaries, and sets; apply slicing to extract sub-sequences; and write basic list/dict/set comprehensions to replace simple loops.

Session Timeline

0:00 – 0:20	Lists	Creation, indexing, mutation, common methods
0:20 – 0:35	Slicing	Slice syntax on lists and strings
0:35 – 0:55	Tuples	Immutability, packing/unpacking, use cases
0:55 – 1:05	■ Break	Short breather
1:05 – 1:35	Dictionaries	Key-value pairs, methods, iteration
1:35 – 1:55	Sets & Comprehensions	Set operations, list/dict/set comprehensions
1:55 – 2:00	Wrap-up	Recap + preview Day 7

■ TIP

Data structures are the single biggest jump in "programmer thinking" so far. Slow down on the list section — everything else builds on the mental model students form here.

Instructor Note

Keep drawing the four structures on the whiteboard side-by-side (list, tuple, dict, set) and ask "which one would you use for X?" throughout the session — this repeated comparison is what makes the choice stick.

2 · Prerequisites & Preparation

Requirements

Completed	Day 5 — Functions & Scope
Python version	3.12 (verify with `python3 --version`)
Editor	VS Code with Python + Pylance extensions active
Environment	Virtual environment activated (`uv venv` from Day 1)

Instructor Pre-Session Checklist

■ Open a fresh VS Code window with an empty `day06/` folder

■ Pre-create four blank files: lists.py, tuples.py, dictionaries.py, sets_comprehensions.py

■ Have a grocery-list / shopping-cart analogy ready for lists

■ Have a GPS-coordinate analogy ready for tuples (fixed, ordered pair)

■ Have a phonebook analogy ready for dictionaries (name → number)

■ Test that `python3 day06_01_lists.py` runs cleanly before class

> **■ TIP**
>
> Print or screen-share a simple table comparing the four structures (ordered? mutable? duplicates allowed? syntax) — students will refer back to it constantly for the rest of the course.

3 · Step-by-Step Instruction Guide

STEP 1 — Lists: Creation, Mutation & Methods

What to Say

"Imagine Priya's grocery list for the week — rice, dal, milk, vegetables. That's exactly what a Python list is: an ordered, changeable collection. Unlike a single variable, a list can hold many items, and we can add, remove, or reorder them any time."

```
# Creating and using a list
grocery_list = ['rice', 'dal', 'milk', 'vegetables']
print(grocery_list)
   ['rice', 'dal', 'milk', 'vegetables']

# Indexing and mutation
grocery_list[0] = 'basmati rice'
grocery_list.append('sugar')
grocery_list.remove('milk')
print(grocery_list)
   ['basmati rice', 'dal', 'vegetables', 'sugar']

# Common methods
grocery_list.sort()
grocery_list.insert(1, 'ghee')
len(grocery_list)
   5
```

■ WATCH OUT

Common student error: confusing `.append()` (adds one item) with `.extend()` (merges another iterable). Show both side by side on the whiteboard.

■ DONE WHEN

Students can create a list, add three items with `.append()`, remove one with `.remove()`, and correctly predict `len()` before running the code.

STEP 2 — Slicing Syntax

What to Say

"Slicing lets us cut out a piece of a list or string using `list[start:stop:step]`. Think of it as telling Python: start here, stop before here, and skip this many each time."

```
# Slicing a list of Mumbai localities
localities = ['Andheri', 'Bandra', 'Colaba', 'Dadar', 'Worli']
localities[1:3]
   ['Bandra', 'Colaba']
localities[:2]
   ['Andheri', 'Bandra']
localities[-2:]
   ['Dadar', 'Worli']
localities[  -1]      # reverse the list
   ['Worli', 'Dadar', 'Colaba', 'Bandra', 'Andheri']

# The same syntax works on strings
city = 'Mumbai'
city[1:4]
   'umb'
```

■ WATCH OUT

The stop index is EXCLUSIVE. `localities[1:3]` gives indices 1 and 2, not 3. This trips up nearly every student the first time — pause here.

STEP 3 — Tuples: Immutability & Unpacking

What to Say

"A tuple looks like a list but with round brackets, and once created it cannot change. Use it for things that should stay fixed — like GPS coordinates or a birth date."

```
# Creating a tuple
coordinates = (19.0760, 72.8777)   # Mumbai lat/long
coordinates[0]
   19.076

# Tuples are immutable
coordinates[0] = 20.0
   TypeError: 'tuple' object does not support item assignment

# Packing and unpacking
student = ('Arjun', 21, 'Delhi')
name, age, city = student
print(name, age, city)
   Arjun 21 Delhi
```

■ DONE WHEN

Students can explain, in their own words, one real situation where a tuple is the better choice over a list, and successfully unpack a 3-item tuple into three variables.

STEP 4 — Dictionaries: Key-Value Pairs

What to Say

"A dictionary is like a phonebook — you look up a name (the key) to get a number (the value). Instead of position, we access items by a unique key."

```
# Creating and using a dictionary
student_marks = {'Riya': 88, 'Sanjay': 74, 'Priya': 91}
student_marks['Riya']
   88

# Adding, updating, removing
student_marks['Arjun'] = 67
student_marks['Sanjay'] = 79
del student_marks['Priya']
print(student_marks)
   {'Riya': 88, 'Sanjay': 79, 'Arjun': 67}

# Common methods
student_marks.keys()
   dict_keys(['Riya', 'Sanjay', 'Arjun'])
student_marks.values()
   dict_values([88, 79, 67])
for name, marks in student_marks.items():
...     print(name, marks)
   Riya 88
   Sanjay 79
   Arjun 67
```

■ WATCH OUT

Common error: using `student_marks['Vikram']` when the key doesn't exist raises a `KeyError`. Introduce `.get('Vikram', 0)` as the safe alternative.

STEP 5 — Sets & Comprehensions

What to Say

"A set stores unique items only — no duplicates — and supports mathematical operations like union and intersection. Comprehensions are a compact way to build a list, dict, or set from a loop in a single line."

```
# Sets — unique collections
mumbai_visitors = {'Priya', 'Arjun', 'Priya', 'Riya'}
print(mumbai_visitors)
   {'Priya', 'Arjun', 'Riya'}

# Set operations
delhi_visitors = {'Riya', 'Sanjay'}
mumbai_visitors & delhi_visitors     # intersection
   {'Riya'}
mumbai_visitors | delhi_visitors     # union
   {'Priya', 'Arjun', 'Riya', 'Sanjay'}

# List comprehension
squares = [n ** 2 for n in range(1, 6)]
   [1, 4, 9, 16, 25]

# Dict comprehension
price_in_rupees = {'tea': 15, 'coffee': 25}
doubled = {k: v * 2 for k, v in price_in_rupees.items()}
   {'tea': 30, 'coffee': 50}

# Set comprehension
even_set = {n for n in range(10) if n % 2 == 0}
   {0, 2, 4, 6, 8}
```

■ DONE WHEN

Students write a one-line list comprehension that filters a list of prices in rupees to only those above ■500, without instructor help.

Common Errors — Quick Reference

TypeError: 'tuple' object does not support item assignment	Trying to modify a tuple	Use a list instead, or rebuild the tuple
KeyError: 'x'	Accessing a missing dict key	Use `.get(key, default)`
IndexError: list index out of range	Index beyond list length	Check `len()` first or use slicing
TypeError: unhashable type: 'list'	Using a list as a dict key or set item	Use a tuple instead of a list

4 · Session Wrap-Up

Student Checklist

■ I can create, index, and modify a list

■ I can slice a list or string with start:stop:step

■ I understand why tuples are immutable and when to use them

■ I can create a dictionary and safely look up values with `.get()`

■ I can perform union/intersection on two sets

■ I can write a simple list comprehension

Key Concepts Recap

List	Yes	Yes	Yes	[1, 2, 3]
Tuple	Yes	No	Yes	(1, 2, 3)
Dictionary	Yes (3.7+)	Yes	Keys: No	{'a': 1}
Set	No	Yes	No	{1, 2, 3}

Next Day Preview

Day 7 moves into Strings & String Methods — formatting, f-strings, `split()`/`join()`, `strip()`/`replace()`, and an introduction to the `re` module. The list/dict comprehension skills from today carry directly into string-processing patterns.

Troubleshooting Notes

If a student's list mutation appears to affect a variable they didn't touch, check whether they copied a list with `b = a` (reference copy) instead of `b = a.copy()` or `b = a[:]` (real copy). This is a very common Day 6 bug and worth a two-minute detour on the whiteboard.

5 · Quick Reference Card

Daily Workflow Commands

```
# Activate environment
source .venv/bin/activate      # macOS/Linux
.venv\Scripts\activate         # Windows

# Run today's files
python3 day06_01_lists.py
python3 day06_02_tuples.py
python3 day06_03_dictionaries.py
python3 day06_04_sets_comprehensions.py
```

VS Code Shortcuts

Ctrl/Cmd + `	Toggle integrated terminal
F5	Run/debug current file
Ctrl/Cmd + /	Toggle line comment
Shift + Alt + F	Format document

Tool Summary

.append() / .extend()	Add one item / merge another iterable into a list
list[start:stop:step]	Slice a list or string
.get(key, default)	Safe dictionary lookup
set1 & set2 / set1 | set2	Set intersection / union
[expr for x in iterable]	List comprehension